Teach Module 34 README

Module 34: State and Event Management in React

This module focuses on how React components remember information, respond to user actions, and update the interface automatically.

Topics covered:

- `useState`
- `useEffect`
- Event handling
- UI reactivity
- Form management
- Performance considerations

Core idea:

> *Events cause state changes, and state changes cause UI updates.*

1. React State

React state lets a component remember data.

Without state, a component can display information, but it cannot meaningfully change based on user interaction. With state, React can re-render the component whenever the stored value changes.

Example:

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  function increase() {
    setCount(count + 1);
  }

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={increase}>Increase</button>
    </div>
  );
}
```

In this line:

```
const [count, setCount] = useState(0);
```

- `count` is the current state value.
- `setCount` is the function used to update that value.

- `0` is the initial value.

When `setCount` is called, React updates the state and re-renders the component.

2. Event Handling

Events are user actions such as:

- clicking a button
- typing in a field
- submitting a form
- selecting an option
- hovering over an element

React event names use camelCase.

Correct:

```
<button onClick={handleClick}>Save</button>
```

Incorrect:

```
<button onclick="handleClick()">Save</button>
```

Another common mistake:

```
<button onClick={handleClick()}>Save</button>
```

That calls the function immediately during rendering.

Use this instead:

```
<button onClick={handleClick}>Save</button>
```

3. Form State

Forms usually need state because React must track what the user types.

Example:

```
import { useState } from "react";

function LoginForm() {
  const [email, setEmail] = useState("");

  function handleChange(event) {
    setEmail(event.target.value);
  }

  function handleSubmit(event) {
    event.preventDefault();
```

```
    console.log("Submitting:", email);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={email}
        onChange={handleChange}
        placeholder="Enter email"
      />

      <button type="submit">Login</button>
    </form>
  );
}
```

Important parts:

```
value={email}
```

The input displays the React state value.

```
onChange={handleChange}
```

Every time the user types, React updates the state.

This is called a controlled component because React controls the input value.

4. useEffect

`useEffect` lets a component perform side effects after rendering.

Common uses:

- loading data
- updating the browser title
- setting timers
- reacting to state changes
- subscribing to external services
- cleaning up resources

Example:

```
import { useEffect, useState } from "react";

function PageTitleCounter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `Count is ${count}`;
  }, [count]);
```

```
  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

This means:

> *Whenever `count` changes, run the effect.*

Common patterns:

```
useEffect(() => {
  // Runs after every render.
});
```

```
useEffect(() => {
  // Runs once when the component first appears.
}, []);
```

```
useEffect(() => {
  // Runs whenever count changes.
}, [count]);
```

5. UI Reactivity

React UI should be based on state.

Instead of manually changing the DOM, update state and let React update the page.

Example:

```
import { useState } from "react";

function ToggleMessage() {
  const [visible, setVisible] = useState(false);

  return (
    <div>
      <button onClick={() => setVisible(!visible)}>
        Toggle
      </button>

      {visible && <p>Hello, React state!</p>}
    </div>
  );
}
```

When `visible` is `true`, the paragraph appears.

When `visible` is `false`, the paragraph disappears.

6. Performance Considerations

Most React apps do not need heavy optimization at first.

A good rule:

> *Do not store values in state if they can be calculated from existing state or props.*

Avoid this:

```
const [fullName, setFullName] = useState("");

useEffect(() => {
  setFullName(firstName + " " + lastName);
}, [firstName, lastName]);
```

Prefer this:

```
const fullName = firstName + " " + lastName;
```

State should represent data that changes over time and cannot simply be derived.

Use tools like `useMemo` only when an expensive calculation is causing a real performance problem.

Practice Exercises

Exercise 1: Counter

Build a counter with three buttons:

- Increase
- Decrease
- Reset

Practice:

- `useState`
- click events
- updating UI from state

Extra challenge: do not allow the counter to go below `0`.

Exercise 2: Show/Hide Message

Create a button that toggles a message.

Practice:

- boolean state
- conditional rendering

- event handling

Exercise 3: Character Counter

Create a text area where the user types a message.

Show:

```
Characters typed: 25
```

Practice:

- form input state
- `onChange`
- controlled components

Extra challenge: set a maximum of `100` characters and show a warning when the user gets close.

Exercise 4: Login Form

Create a simple login form with:

- Email
- Password
- Submit button

When the user submits, show:

```
Welcome, user@example.com
```

Practice:

- form state
- `event.preventDefault()`
- submit events
- basic validation

Extra challenge: show an error if email or password is empty.

Exercise 5: Todo List

Build a small todo app.

Features:

- add a todo
- display todos
- mark a todo complete
- delete a todo

Practice:

- array state
- event handling
- rendering lists with `.map()`
- updating objects inside state

Exercise 6: Color Picker

Create buttons for different colors:

- Red
- Blue
- Green
- Yellow

When a button is clicked, change the background color of a box.

Practice:

- state-driven styling
- click events
- dynamic CSS

Exercise 7: Product Quantity Selector

Create a product card with this behavior:

```
Product: JavaScript Book
Price: $25
Quantity: [-] 1 [+]
Total: $25
```

Practice:

- numeric state
- derived values
- preventing invalid state

Extra challenge: disable the minus button when quantity is `1`.

Exercise 8: Live Search Filter

Create a list of names or products.

Add a search box. As the user types, filter the list.

Practice:

- input state
- derived UI
- `.filter()`
- re-rendering based on state

Example list:

```
["Java", "Spring Boot", "React", "Oracle", "REST API"]
```

Exercise 9: useEffect Page Title

Create a counter and update the browser tab title whenever the count changes.

Practice:

- `useEffect`
- dependency array
- side effects

Example:

```
useEffect(() => {
  document.title = `Count: ${count}`;
}, [count]);
```

Exercise 10: Form With Multiple Fields

Create a registration form with:

- First Name
- Last Name
- Email
- Role

Show a live preview below the form.

Practice:

- object state
- multiple input handlers
- controlled forms

Example state:

```
const [formData, setFormData] = useState({
  firstName: "",
  lastName: "",
  email: "",
  role: ""
});
```

Module 34 Lab: Task Tracker App

Goal

Build a small Task Tracker App in React where a user can add tasks, mark them complete, delete them, and filter what they see.

Features

Build the following:

1. Add a new task using an input field and button.
2. Display all tasks in a list.
3. Mark a task as complete or incomplete.
4. Delete a task.
5. Show task count.
6. Filter tasks by All, Active, and Completed.

7. Clear the input after adding a task.
8. Prevent empty tasks from being added.

Starter Component

```
import { useState } from "react";

function TaskTracker() {
  const [taskText, setTaskText] = useState("");
  const [tasks, setTasks] = useState([]);
  const [filter, setFilter] = useState("all");

  function handleAddTask(event) {
    event.preventDefault();

    if (taskText.trim() === "") {
      return;
    }

    const newTask = {
      id: Date.now(),
      text: taskText,
      completed: false
    };

    setTasks([...tasks, newTask]);
    setTaskText("");
  }

  function handleToggleTask(id) {
    setTasks(
      tasks.map((task) =>
        task.id === id
          ? { ...task, completed: !task.completed }
          : task
      )
    );
  }

  function handleDeleteTask(id) {
    setTasks(tasks.filter((task) => task.id !== id));
  }

  const filteredTasks = tasks.filter((task) => {
    if (filter === "active") {
      return !task.completed;
    }

    if (filter === "completed") {
      return task.completed;
    }
```

```
    return true;
  });

  return (
    <div>
      <h2>Task Tracker</h2>

      <form onSubmit={handleAddTask}>
        <input
          type="text"
          value={taskText}
          onChange={(event) => setTaskText(event.target.value)}
          placeholder="Enter a task"
        />

        <button type="submit">Add Task</button>
      </form>

      <div>
        <button onClick={() => setFilter("all")}>All</button>
        <button onClick={() => setFilter("active")}>Active</button>
        <button onClick={() => setFilter("completed")}>Completed</button>
      </div>

      <p>Total tasks: {tasks.length}</p>

      <ul>
        {filteredTasks.map((task) => (
          <li key={task.id}>
            <span
              onClick={() => handleToggleTask(task.id)}
              style={{
                textDecoration: task.completed ? "line-through" : "none",
                cursor: "pointer"
              }}
            >
              {task.text}
            </span>

            <button onClick={() => handleDeleteTask(task.id)}>
              Delete
            </button>
          </li>
        ))}
      </ul>
    </div>
  );
}

export default TaskTracker;
```

Lab Tasks

1. Create a new React component named `TaskTracker`.
2. Add the starter code.
3. Render it from `App.jsx`.
4. Test adding tasks.
5. Test marking tasks complete.
6. Test deleting tasks.
7. Test each filter button.
8. Add a message when no tasks exist.

Example empty-state message:

```
{tasks.length === 0 && <p>No tasks yet.</p>}
```

Challenge Extensions

Add these after the main lab works:

1. Show active task count.
2. Disable the Add button when input is empty.
3. Add an edit task feature.
4. Add a priority dropdown: Low, Medium, High.
5. Use `useEffect` to update the browser title.

Example:

```
useEffect(() => {
  document.title = `${tasks.length} tasks`;
}, [tasks]);
```

Quick Review

- `useState` stores component data.
- Events are user actions such as clicks, typing, and form submissions.
- Controlled inputs use React state as their value.
- `useEffect` handles side effects after rendering.
- React updates the UI when state changes.
- Derived values usually do not need their own state.
- Arrays in state should be updated immutably with methods like `map`, `filter`, and spread syntax.